kNN 데이터 분석

임광윤

2025-01-11

Table of Contents

Q1: 데이터 로드 및 결측치 처리 1

Q2: 데이터 분할 (Train/Validation/Test) 2

Q3: kNN 분류 (k=3) 3

Q4: 최적의 k 찾기 4

Q5: Test 데이터에 대한 kNN 분류 6

Q1: 데이터 로드 및 결측치 처리

설명

- `PimaIndiansDiabetes2` 데이터셋을 로드하고 결측치를 확인한 후 제거합니다.

```r
# 필요한 패키지 설치 및 로드
if (!requireNamespace("mlbench", quietly = TRUE)) {
  install.packages("mlbench")
}
library(mlbench)

# 데이터 로드 및 구조 확인
data("PimaIndiansDiabetes2")
str(PimaIndiansDiabetes2)  # 데이터 구조 확인

## 'data.frame':    768 obs. of  9 variables:
##  $ pregnant: num  6 1 8 1 0 5 3 10 2 8 ...
##  $ glucose : num  148 85 183 89 137 116 78 115 197 125 ...
##  $ pressure: num  72 66 64 66 40 74 50 NA 70 96 ...
##  $ triceps : num  35 29 NA 23 35 NA 32 NA 45 NA ...
##  $ insulin : num  NA NA NA 94 168 NA 88 NA 543 NA ...
```

```
##  $ mass    : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 NA ...
##  $ pedigree: num  0.627 0.351 0.672 0.167 2.288 ...
##  $ age     : num  50 31 32 21 33 30 26 29 53 54 ...
##  $ diabetes: Factor w/ 2 levels "neg","pos": 2 1 2 1 2 1 2 1 2 2 ...
```

```r
# 결측치 확인
colSums(is.na(PimaIndiansDiabetes2))
```

```
## pregnant  glucose pressure  triceps  insulin     mass pedigree      age
##        0        5       35      227      374       11        0        0
## diabetes
##        0
```

결측치 제거

```r
# 결측치 제거
cleaned_data <- na.omit(PimaIndiansDiabetes2)
str(cleaned_data)  # 결측치 제거 후 데이터 구조 확인
```

```
## 'data.frame':    392 obs. of  9 variables:
##  $ pregnant: num  1 0 3 2 1 5 0 1 1 3 ...
##  $ glucose : num  89 137 78 197 189 166 118 103 115 126 ...
##  $ pressure: num  66 40 50 70 60 72 84 30 70 88 ...
##  $ triceps : num  23 35 32 45 23 19 47 38 30 41 ...
##  $ insulin : num  94 168 88 543 846 175 230 83 96 235 ...
##  $ mass    : num  28.1 43.1 31 30.5 30.1 25.8 45.8 43.3 34.6 39.3 ...
##  $ pedigree: num  0.167 2.288 0.248 0.158 0.398 ...
##  $ age     : num  21 33 26 53 59 51 31 33 32 27 ...
##  $ diabetes: Factor w/ 2 levels "neg","pos": 1 2 2 2 2 2 2 1 2 1 ...
##  - attr(*, "na.action")= 'omit' Named int [1:376] 1 2 3 6 8 10 11 12 13 16
 ...
##   ..- attr(*, "names")= chr [1:376] "1" "2" "3" "6" ...
```

Q2: 데이터 분할 (Train/Validation/Test)

설명

- 데이터를 3:1:1 비율로 학습, 검증, 테스트 세트로 나누고 독립 변수와 종속 변수로 분리합니다.

```r
# 데이터 분할
set.seed(123)  # 재현성을 위한 난수 고정
n <- nrow(cleaned_data)
train_size <- round(0.6 * n)
valid_size <- round(0.2 * n)
test_size <- n - train_size - valid_size
```

```r
indices <- sample(1:n)
train_indices <- indices[1:train_size]
valid_indices <- indices[(train_size + 1):(train_size + valid_size)]
test_indices <- indices[(train_size + valid_size + 1):n]

train_data <- cleaned_data[train_indices, ]
valid_data <- cleaned_data[valid_indices, ]
test_data <- cleaned_data[test_indices, ]

# 독립 변수와 종속 변수 분리
train_x <- train_data[, -9]
train_y <- train_data[, 9]
valid_x <- valid_data[, -9]
valid_y <- valid_data[, 9]
test_x <- test_data[, -9]
test_y <- test_data[, 9]
```

Q3: kNN 분류 (k=3)

설명

- 검증 세트에 대해 k=3 으로 설정한 후 kNN 분류를 수행하고, 정확도와 예측 결과를 확인합니다.

```r
# 필요한 패키지 설치 및 로드
if (!requireNamespace("FNN", quietly = TRUE)) {
  install.packages("FNN")
}
library(FNN)

## Warning: 패키지 'FNN'는 R 버전 4.4.2 에서 작성되었습니다

# k=3 분류 수행
knn_pred <- knn(
  train = as.matrix(train_x),
  test = as.matrix(valid_x),
  cl = train_y,
  k = 3
)

# 정확도 계산
accuracy_q3 <- sum(knn_pred == valid_y) / length(valid_y)
cat("Q3: k=3 일 때 검증 세트의 정확도:", accuracy_q3, "\n")
```

```
## Q3: k=3 일 때 검증 세트의 정확도: 0.6153846
```

```r
# 예측 값과 실제 값 비교
comparison_q3 <- data.frame(
  Actual = valid_y,
  Predicted = knn_pred
)
head(comparison_q3)  # 일부 결과 출력
```

```
##   Actual Predicted
## 1    pos       neg
## 2    neg       neg
## 3    neg       neg
## 4    pos       pos
## 5    neg       pos
## 6    pos       pos
```

Q4: 최적의 k 찾기

설명

- 10-폴드 교차 검증을 사용하여 최적의 k 값을 탐색하고, 결과를 시각화합니다.

데이터 표준화 및 모델 학습

```r
# 필요한 패키지 설치 및 로드
if (!requireNamespace("caret", quietly = TRUE)) {
  install.packages("caret")
}
library(caret)
```

```
## Warning: 패키지 'caret'는 R 버전 4.4.2 에서 작성되었습니다

## 필요한 패키지를 로딩중입니다: ggplot2

## Warning: 패키지 'ggplot2'는 R 버전 4.4.2 에서 작성되었습니다

## 필요한 패키지를 로딩중입니다: lattice
```

```r
# 데이터 표준화
preproc <- preProcess(train_x, method = c("center", "scale"))
train_x_norm <- predict(preproc, train_x)
valid_x_norm <- predict(preproc, valid_x)
```

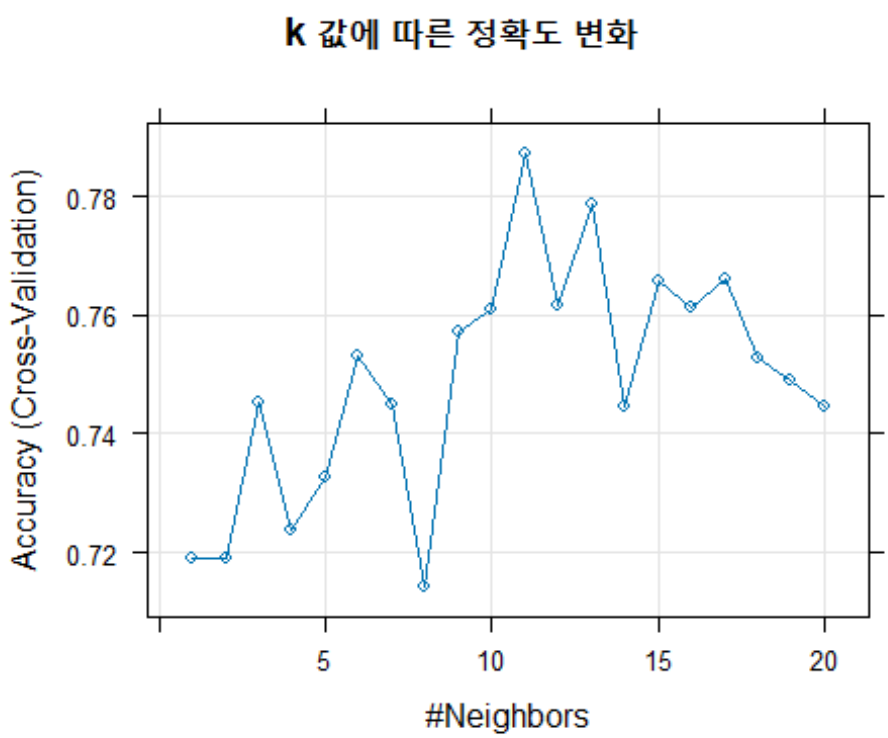
```r
# 최적의 k 값 찾기
control <- trainControl(method = "cv", number = 10)  # 10-폴드 교차 검증
model <- train(
  x = train_x_norm,
  y = train_y,
  method = "knn",
  trControl = control,
  tuneGrid = expand.grid(k = 1:20)  # k 값 탐색 범위
)

# 최적의 k 값 출력
optimal_k <- model$bestTune$k
optimal_accuracy <- max(model$results$Accuracy)
cat("Q4: 최적의 k:", optimal_k, "\n 최적의 정확도:", optimal_accuracy, "\n")

## Q4: 최적의 k: 11
## 최적의 정확도: 0.7871377
```

결과 시각화

```r
plot(model, main = "k 값에 따른 정확도 변화")
```



Q5: Test 데이터에 대한 kNN 분류

설명

- 최적의 k 값을 사용하여 테스트 세트를 예측하고, 정확도와 Confusion Matrix 를 계산합니다.

```r
# 최적의 k 를 사용한 테스트 세트 분류
test_pred <- knn(
  train = as.matrix(train_x),
  test = as.matrix(test_x),
  cl = train_y,
  k = optimal_k
)

# 테스트 세트 정확도 계산
accuracy_q5 <- sum(test_pred == test_y) / length(test_y)
cat("Q5: 최적의 k=", optimal_k, "에서 테스트 세트의 정확도:", accuracy_q5, "\n")
```

```
## Q5: 최적의 k= 11 에서 테스트 세트의 정확도: 0.8481013
```

```r
# 예측 값과 실제 값 비교
comparison_q5 <- data.frame(
  Actual = test_y,
  Predicted = test_pred
)
head(comparison_q5)  # 일부 결과 출력
```

```
##   Actual Predicted
## 1    neg       neg
## 2    neg       neg
## 3    pos       pos
## 4    neg       neg
## 5    pos       neg
## 6    neg       neg
```

```r
# Confusion Matrix
confusion_q5 <- confusionMatrix(factor(test_pred), factor(test_y))
print(confusion_q5)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction neg pos
##        neg  50   6
##        pos   6  17
```

```
##
##                Accuracy : 0.8481
##                  95% CI : (0.7497, 0.919)
##     No Information Rate : 0.7089
##     P-Value [Acc > NIR] : 0.003098
##
##                   Kappa : 0.632
##
##  Mcnemar's Test P-Value : 1.000000
##
##             Sensitivity : 0.8929
##             Specificity : 0.7391
##          Pos Pred Value : 0.8929
##          Neg Pred Value : 0.7391
##              Prevalence : 0.7089
##          Detection Rate : 0.6329
##    Detection Prevalence : 0.7089
##       Balanced Accuracy : 0.8160
##
##        'Positive' Class : neg
##
```